Turing Function Calling Assessment Guide

Complete guide to understanding and acing the Turing Function Calling Test for AI training roles.

What is Function Calling?

Function calling = Ability of LLMs to identify and execute specific functions/tools based on user requests

Key Concept: LLM decides:

1. **Which function** to call (tool selection)
2. **What parameters** to pass (payload construction)

Example:

```
User: "What's the weather in London?"

LLM identifies:
- Function: get_weather()
- Payload: {"location": "London", "unit": "celsius"}
```

How Function Calling Works

Step 1: Define Available Functions

```
tools = [
    {
        "type": "function",
        "function": {
            "name": "get_weather",
            "description": "Get current weather for a location",
            "parameters": {
                "type": "object",
                "properties": {
                    "location": {
                        "type": "string",
                        "description": "City name or zip code"
                    },
                    "unit": {
                        "type": "string",
                        "enum": ["celsius", "fahrenheit"],
                        "description": "Temperature unit"
                    }
                },
                "required": ["location"]
            }
        }
```

```
    },
    {
        "type": "function",
        "function": {
            "name": "search_database",
            "description": "Search for records in database",
            "parameters": {
                "type": "object",
                "properties": {
                    "query": {
                        "type": "string",
                        "description": "Search query"
                    },
                    "table": {
                        "type": "string",
                        "enum": ["users", "orders", "products"],
                        "description": "Database table to search"
                    }
                },
                "required": ["query", "table"]
            }
        }
    }
]
```

Step 2: User Makes Request

```
User: "What's the weather in Tokyo? Give it to me in Fahrenheit."
```

Step 3: LLM Identifies Function and Payload

```
{
  "function_name": "get_weather",
  "arguments": {
    "location": "Tokyo",
    "unit": "fahrenheit"
  }
}
```

Step 4: Execute Function

```
result = get_weather(location="Tokyo", unit="fahrenheit")
# Returns: {"temperature": 68, "condition": "Sunny", "humidity": 60}
```

Step 5: LLM Formats Response

```
LLM: "The current weather in Tokyo is 68°F and sunny with 60% humidity."
```

Test Format (What to Expect)

Type 1: Tool Selection

Given: User query + Available functions

Task: Identify which function to call

Example:

```
User: "Show me all orders from customer John Doe"

Available functions:
1. get_user(name)
2. search_orders(customer_name)
3. get_product(product_id)
4. send_email(to, subject, body)

Correct answer: search_orders
```

Why: Query asks for orders, only `search_orders` retrieves order data.

Type 2: Payload Construction

Given: User query + Function schema

Task: Construct correct JSON payload

Example:

```
User: "Book a flight from NYC to LAX on December 25th, economy class"

Function: book_flight(origin, destination, date, class)

Correct payload:
{
  "origin": "NYC",
  "destination": "LAX",
  "date": "2024-12-25",
  "class": "economy"
}
```

Type 3: Multi-Step Function Calling

Given: Complex query requiring multiple functions

Task: Identify sequence of functions and payloads

Example:

```
User: "Find John Doe's email and send him a reminder about his pending order"

Step 1: get_user(name="John Doe") → Returns {"email": "john@example.com", "id":
123}
Step 2: get_pending_orders(user_id=123) → Returns order details
Step 3: send_email(to="john@example.com", subject="Order Reminder", body="...")
```

Type 4: Error Handling

Given: Ambiguous or invalid request

Task: Identify missing information or suggest clarification

Example:

```
User: "Book a flight to Paris"

Issue: Missing required fields (origin, date)

Correct response: Ask for clarification
"I need more information to book your flight. Could you provide:
- Departure city
- Travel date
- Preferred class (economy/business/first)"
```

Type 5: Parameter Extraction

Given: Natural language with implicit parameters

Task: Extract and format correctly

Example:

```
User: "What was the temperature in SF yesterday?"

Challenges:
- "SF" → "San Francisco" (expand abbreviation)
- "yesterday" → "2024-08-04" (calculate date)

Correct payload:
{
  "location": "San Francisco",
```

```
  "date": "2024-08-04"
}
```

Common Test Scenarios

Scenario 1: Basic Tool Selection

```
Available tools:
- calculate(expression: string)
- get_stock_price(symbol: string)
- send_sms(phone: string, message: string)

Q1: "What's 25 * 37?"
Answer: calculate, {"expression": "25 * 37"}

Q2: "What's the current price of Apple stock?"
Answer: get_stock_price, {"symbol": "AAPL"}

Q3: "Text my boss that I'll be late"
Answer: send_sms - but MISSING required field (phone number)
Correct: Ask for phone number
```

Scenario 2: Nested Objects

```
Function: create_user(
  name: string,
  contact: {
    email: string,
    phone: string
  },
  address: {
    street: string,
    city: string,
    zip: string
  }
)

User: "Create account for Alice Smith, email alice@example.com,
       phone 555-1234, living at 123 Main St, Boston, MA 02101"

Correct payload:
{
  "name": "Alice Smith",
  "contact": {
    "email": "alice@example.com",
    "phone": "555-1234"
  },
  "address": {
```

```
    "street": "123 Main St",
    "city": "Boston",
    "zip": "02101"
  }
}
```

Scenario 3: Arrays/Lists

```
Function: add_items_to_cart(user_id: int, items: array)

User: "Add milk, eggs, and bread to cart for user 123"

Correct payload:
{
  "user_id": 123,
  "items": ["milk", "eggs", "bread"]
}
```

Scenario 4: Enums/Fixed Values

```
Function: set_thermostat(temperature: int, mode: enum["heat", "cool", "auto"])

User: "Set thermostat to 72 degrees for cooling"

Correct payload:
{
  "temperature": 72,
  "mode": "cool"
}

Wrong: {"mode": "cooling"} - Must use exact enum value
```

Scenario 5: Optional Parameters

```
Function: search_products(
  query: string (required),
  category: string (optional),
  min_price: float (optional),
  max_price: float (optional)
)

User: "Find laptops under $1000"

Correct payload:
{
  "query": "laptops",
```

```
  "max_price": 1000.0
}

Note: Don't include optional params if not mentioned
```

Practice Questions

Set 1: Tool Selection

Available Tools:

```
1. get_user_profile(user_id: int)
2. update_user_email(user_id: int, email: string)
3. delete_user(user_id: int)
4. list_all_users(limit: int)
5. search_users(query: string)
```

Questions:

Q1: "Show me details for user 456"

▶ Answer

Q2: "Change user 123's email to newemail@example.com"

▶ Answer

Q3: "Find all users named John"

▶ Answer

Q4: "Get the first 50 users"

▶ Answer

Set 2: Complex Payloads

Tool:

```
book_hotel(
    location: string,
    check_in: string,  # Format: YYYY-MM-DD
    check_out: string,  # Format: YYYY-MM-DD
    guests: int,
    rooms: int,
    preferences: {
        room_type: enum["single", "double", "suite"],
        amenities: array[string]  # Optional
```

```
    }
)
```

Q5: "Book a hotel in Paris for 2 guests from Dec 20-25, 2024. We need a suite with a pool and gym."

▶ Answer

Q6: "Reserve a room in Tokyo for Dec 1st to Dec 5th, single room, 1 person"

▶ Answer

Set 3: Ambiguous Requests

Tool:

```
transfer_money(
    from_account: string,  # Required
    to_account: string,    # Required
    amount: float,         # Required
    currency: string,      # Required
    memo: string          # Optional
)
```

Q7: "Transfer $500 to account ABC123"

▶ Answer

Correct response: "I need more information. Which account should I transfer from?"

Cannot proceed without required field.

Q8: "Send 100 euros from my savings to account XYZ789 for rent payment"

▶ Answer

Set 4: Multi-Step Reasoning

Available Tools:

```
1. get_order(order_id: int)
2. get_customer(customer_id: int)
3. cancel_order(order_id: int, reason: string)
4. send_notification(customer_id: int, message: string)
```

Q9: "Cancel order 789 due to stock shortage and notify the customer"

▶ Answer

Three function calls needed in sequence.

Set 5: Type Conversions

Tool:

```
schedule_meeting(
    title: string,
    date: string,        # YYYY-MM-DD
    time: string,        # HH:MM (24-hour)
    duration: int,       # minutes
    attendees: array[string]  # email addresses
)
```

Q10: "Schedule a team sync tomorrow at 2:30 PM for 90 minutes with alice@company.com and bob@company.com"

▶ Answer

Key conversions:

- "tomorrow" → calculate actual date
- "2:30 PM" → "14:30"
- "90 minutes" → 90 (integer)

Common Mistakes to Avoid

1. Wrong Data Type

```
# Function expects: amount: float
# User: "Transfer 50 dollars"

❌ Wrong: {"amount": "50"}     # String
☑ Correct: {"amount": 50.0}   # Float
```

2. Missing Required Fields

```
# Function: send_email(to: string, subject: string, body: string)
# All fields required

❌ Wrong: {"to": "user@example.com", "subject": "Hello"}  # Missing body
☑ Correct: {"to": "user@example.com", "subject": "Hello", "body": "Hi there"}
```

3. Including Unnecessary Fields

```
# Function only accepts: name, email

❌ Wrong: {"name": "Alice", "email": "alice@example.com", "age": 30}
☑ Correct: {"name": "Alice", "email": "alice@example.com"}
```

4. Incorrect Enum Values

```
# Function: set_status(status: enum["pending", "approved", "rejected"])

❌ Wrong: {"status": "approve"}     # Not exact enum value
☑ Correct: {"status": "approved"}  # Exact match
```

5. Not Handling Ambiguity

```
# User: "Set temperature to 72"

# Function needs: temperature + mode (heat/cool/auto)

❌ Wrong: Guess the mode
☑ Correct: Ask "Would you like heating or cooling mode?"
```

6. Over-Interpreting Context

```
# User: "Order pizza"

# Function: order_food(item, size, quantity, address)

❌ Wrong: Assume size=large, quantity=1, use user's last address
☑ Correct: Ask for missing required information
```

7. Incorrect Array Format

```
# Function: add_tags(tags: array[string])

❌ Wrong: {"tags": "python, redis, cache"}  # String with commas
☑ Correct: {"tags": ["python", "redis", "cache"]}  # Array
```

Test-Taking Strategy

1. Read Function Schema Carefully

Focus on:

- Required vs optional parameters
- Data types (string, int, float, boolean, array, object)
- Enum values (must match exactly)
- Parameter descriptions
- Default values

2. Extract All Information from User Query

Technique:

1. Highlight key information in the query
2. Map each piece to a function parameter
3. Check if all required fields are covered
4. Don't add information not in the query

Example:

```
User: "Search for hotels in Miami, check-in Dec 15, 2 guests"

Extracted:
- location: "Miami" ✓
- check_in: "Dec 15" → "2024-12-15" ✓
- guests: 2 ✓
- check_out: NOT PROVIDED ❌

Action: Ask for check-out date (if required)
```

3. Convert Natural Language to Correct Format

Common conversions:

- Dates: "tomorrow" → "2026-08-06"
- Times: "3 PM" → "15:00"
- Numbers: "fifty" → 50
- Abbreviations: "NYC" → "New York City"
- Relative: "yesterday" → calculate actual date

4. Handle Missing Information

Decision tree:

```
Is field required?
├─ Yes → Cannot proceed, ask user
└─ No → Check if mentioned in query
   ├─ Yes → Include in payload
   └─ No → Omit from payload (don't assume)
```

5. Validate Your Answer

Checklist:

- ☑ All required fields present?
- ☑ Correct data types?
- ☑ Enum values exact match?
- ☑ No extra fields added?
- ☑ Arrays formatted correctly?
- ☑ Nested objects structured properly?

Advanced Scenarios

1. Conditional Logic

```
# Function: apply_discount(order_id: int, discount: float)
# Discount must be between 0.0 and 1.0 (percentage)

User: "Apply 25% discount to order 123"

Payload:
{
  "order_id": 123,
  "discount": 0.25  # Convert 25% to 0.25
}
```

2. Date/Time Calculations

```
# Function: schedule_task(task: string, due_date: string)

User: "Remind me to submit report in 3 days"

Today: August 5, 2026
Due date: August 5 + 3 = August 8, 2026

Payload:
{
  "task": "submit report",
  "due_date": "2026-08-08"
}
```

3. Multiple Items in One Request

```
# Function: add_to_cart(items: array[{product_id: int, quantity: int}])

User: "Add 2 apples (ID: 101), 3 oranges (ID: 102), and 1 banana (ID: 103)"
```

```
Payload:
{
  "items": [
    {"product_id": 101, "quantity": 2},
    {"product_id": 102, "quantity": 3},
    {"product_id": 103, "quantity": 1}
  ]
}
```

4. Unit Conversions

```
# Function: set_distance(distance: float, unit: enum["km", "miles"])

User: "Set distance to 5000 meters"

Conversion: 5000 meters = 5 km

Payload:
{
  "distance": 5.0,
  "unit": "km"
}
```

5. Resolving References

```
# User: "Send the same message to Bob"
# Previous context: Sent "Hello" to Alice

Must maintain context:
- "same message" refers to "Hello"
- "Bob" is the new recipient

Payload:
{
  "to": "bob@example.com",
  "message": "Hello"
}
```

Real Turing-Style Practice Test

Scenario: Customer Support Bot

Available Functions:

```
1. lookup_order(order_id: int) → Returns order details

2. update_order_status(
     order_id: int,
     status: enum["processing", "shipped", "delivered", "cancelled"]
   )

3. refund_order(
     order_id: int,
     amount: float,
     reason: string
   )

4. send_email(
     to: string,
     subject: string,
     body: string
   )

5. check_inventory(
     product_id: int,
     quantity: int
   ) → Returns {available: boolean}

6. create_support_ticket(
     customer_id: int,
     issue: string,
     priority: enum["low", "medium", "high"]
   )
```

Test Questions:

Q1: "What's the status of order 45678?"

▶ Answer

Q2: "Mark order 12345 as shipped"

▶ Answer

Q3: "Refund order 98765 for $50.75 because item was damaged"

▶ Answer

Q4: "Check if we have 10 units of product 555 in stock"

▶ Answer

Q5: "Customer 789 has a high priority issue with delivery delays"

▶ Answer

Q6: "Cancel order 11111 and email customer@example.com about the cancellation"

▶ Answer

Call 1: Function: update_order_status Payload: {"order_id": 11111, "status": "cancelled"}

Call 2: Function: send_email Payload: { "to": "customer@example.com", "subject": "Order Cancellation", "body": "Your order 11111 has been cancelled." }

Q7: "Process a refund for customer's order"

▶ Answer

Correct response: "I need more information to process the refund. Please provide:

- Order ID
- Refund amount
- Reason for refund"

Tips for Success

Before the Test

1. **Understand JSON structure**
    - Objects: `{}`
    - Arrays: `[]`
    - Strings: `"text"`
    - Numbers: `123` or `123.45`
    - Booleans: `true` / `false`
2. **Practice type conversions**
    - Text to numbers
    - Dates to ISO format
    - Times to 24-hour format
    - Percentages to decimals
3. **Study enum patterns**
    - Must match exactly (case-sensitive)
    - No synonyms (e.g., "approve" ≠ "approved")

During the Test

1. **Read carefully**
    - Understand what user wants
    - Check function schema
    - Note required vs optional fields
2. **Work systematically**
    - Extract all info from query

- Map to function parameters
- Validate completeness

3. **Don't assume**
   - Only use information provided
   - Ask if required fields missing
   - Don't add optional fields not mentioned

4. **Check your work**
   - Syntax valid?
   - All required fields present?
   - Correct data types?
   - Enum values exact match?

Common Test Tricks

Trick 1: Similar function names

```
- get_user(user_id)
- get_user_profile(user_id)  # More detailed
- get_users()  # Returns all users

Choose the most specific function for the task
```

Trick 2: Optional parameters that seem required

```
User: "Search for products"

Function: search_products(query: string, category: string (optional))

Don't add category if not mentioned!
```

Trick 3: Ambiguous pronouns

```
User: "Send him the invoice"

Who is "him"? Need email address.
Correct: Ask for clarification
```

Resources for Practice

Online Practice

1. **OpenAI Function Calling Docs**
    - Examples and patterns
    - https://platform.openai.com/docs/guides/function-calling
2. **Anthropic Claude Tools**
    - Similar concepts
    - https://docs.anthropic.com/claude/docs/tool-use
3. **LangChain Agents**
    - Practice with tool selection
    - https://python.langchain.com/docs/modules/agents/

Practice Exercises

Create your own scenarios:

1. Define 5-10 functions
2. Write user queries
3. Determine correct function + payload
4. Check against schema

Mock Tests

Time yourself:

- 10 questions in 15 minutes
- Focus on accuracy first, then speed
- Review mistakes

Summary

What Turing Tests:

- Tool selection (which function?)
- Payload construction (correct parameters?)
- Handling ambiguity (ask for clarification?)
- Multi-step reasoning (sequence of calls?)

Keys to Success: ☑ Understand JSON structure ☑ Read function schemas carefully ☑ Extract information systematically ☑ Don't assume or add extra info ☑ Validate data types and required fields ☑ Ask for clarification when needed

Practice Focus:

- Type conversions (dates, times, numbers)
- Enum values (exact match)
- Required vs optional fields
- Nested objects and arrays

- Multi-step function calling

Remember: The test evaluates your ability to map natural language to structured function calls accurately. Be precise, don't assume, and validate your answer!